



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE CIENCIAS

Proyecto 1

Chat TCP

Alumno:

Espinosa Meléndez Juan Ernesto
No. de cuenta: 323150066

Asignatura:

Modelado y Programación

Profesor:

Canék Peláez Valdés

Ayudantes:

Cynthia Lizbeth Sánchez Urbano
Miriam Torres Bucio

Índice

1. Introducción	3
2. Análisis y modelado	3
2.1. Análisis	3
2.2. Modelado	4
3. Descripción general del sistema	5
3.1. Arquitectura cliente/servidor	5
4. Protocolo de comunicación	6
4.1. Operaciones del protocolo	6
5. Implementación	6
5.1. Tecnologías utilizadas	6
5.2. Requerimientos	7
5.3. Compilación y ejecución	7
5.4. Organización del proyecto	7
6. Refactorización y patrones de diseño	8
6.1. Factory	8
6.2. Command	8
7. Diagramas UML	8
7.1. Diagrama de componentes	8
7.2. Diagrama de clases	9
7.3. Diagrama de secuencia	9
8. Problemas encontrados y soluciones	10
8.1. Problema 1	10
8.1.1. Solución	10
8.2. Problema 2	10
8.2.1. Solución	10
8.3. Problema 3	10
8.3.1. Solución	10
8.4. Problema 4	10
8.4.1. Solución	10
8.5. Problema 5	11
8.5.1. Solución	11
8.6. Problema 6	11
8.6.1. Solución	11
9. Conclusiones	11
10. Referencias	11

1. Introducción

Este proyecto trata sobre el análisis, modelado y desarrollo de un chat TCP, en donde debíamos investigar como programar un servidor y un cliente que se pudieran comunicar entre si. Además, debíamos determinar en que lenguaje se iba a programar, en este caso se realizó el servidor en C y el cliente en C++, tomando en cuenta que debíamos enviar y recibir de manera interna mensajes JSON que el usuario no iba a poder ver ni escribir, pero en cambio el servidor si debía lidiar con ellos.

Una vez terminadas las bases teníamos un protocolo que se debía cumplir para crear salas privadas y una sala pública, enviar invitaciones y aceptarlas, solicitar la lista de usuarios en las respectivas salas, cambiar el estatus e identificarse. Durante las clases se fueron dando otras indicaciones que complementaron el protocolo, tales fueron los casos de definir un límite al tamaño de mensajes, ya que solo existía un límite en el username que podía utilizar un usuario y un límite en el nombre de las salas privadas. Este proyecto da por hecho que el programa va a funcionar adecuadamente, ya que se cuentan con las bases necesarias, siendo que lo más importante es el análisis, modelado y desarrollo del mismo, sin embargo como se verá más adelante, hubo distintos problemas a la hora del desarrollo y en la cuestión del modelado.

2. Análisis y modelado

2.1. Análisis

La elección del lenguaje fue algo que no pensé demasiado, ya que, Canek ofreció 1 punto extra si hacíamos el servidor en C y el usuario en un lenguaje distinto, así que opté por utilizar C en el servidor y C++ para el usuario, no solo por el punto extra que ofreció Canek, sino que también contaba con libros para aprender a programar en C y C++, implementar algoritmos en C y de programación estructurada en C; con esto como base pensé que sería sencillo utilizar estos 2 lenguajes dado que contaba con bibliografía a la mano que me podía ser de utilidad y además podía aspirar a 2 puntos extra.

Antes de comenzar, busqué distintos tutoriales para saber como se hacía un servidor, adicionalmente leí como funcionaban los sockets porque esto era muy importante para el proyecto y era algo de lo que no tenía ni idea de como implementarlo. Afortunadamente encontré un par de tutoriales que me fueron de gran utilidad al comienzo.

Mi idea al comienzo era realizar una interfaz y que el programa sería parecido al messenger viejito que tenía zumbidos, visualmente me podía dar una idea, sin embargo en la cuestión del servidor lo único que tenía claro era lo que había podido observar en los tutoriales, adicionalmente lo único que sabía de C y C++ era lo que había podido leer en la primera semana.

Lo que pensé para el orden de como desarrollar el proyecto fue: comenzar el servidor hasta que aceptara conexiones y utilizar un usuario de prueba que me permitiera asegurarme que el servidor funcionaba. En este punto pude haber hecho tests para no hacerlo manualmente pero la verdad no tenía idea de como hacerlas y cada vez me quedaba menos tiempo para la fecha de entrega y no tenía nada en mi repositorio más que el archivo de

prueba. Posteriormente traté de hacer que el servidor fuera estable, guardara los clientes en un diccionario, funcionará la lista de usuarios conectados y pudieras cambiar tu estatus, para que una vez terminara con esto fuera de lleno con todo lo que abarcaba el protocolo respecto a salas y enviar mensajes.

El cliente en algún punto tenía que crearlo, pero lo pospuse lo más que pude ya que no había investigado tantas cosas de C++ como lo hice con C, cuando el cliente de prueba me dejó de ser útil lo deseché y comencé con el desarrollo de uno propio que pudiera cumplir con todo lo que el protocolo requería.

Una vez que ya había comenzado, los tutoriales comenzaron a ser escasos y me guíé por repositorios que habían hecho un proyecto similar o igual al que me pedían, sin embargo, algunos no me eran completamente de utilidad, ya que algunos limitaban la cantidad de usuarios, no usaban JSON o estaban hechos para windows. Por tanto los use más como una guía de lo siguiente que debía hacer, lo bueno es que en el caso del cliente al haber escogido C++ contaba con referencias mucho más útiles que para guiarme en el desarrollo del servidor en C.

2.2. Modelado

Traté de programar siguiendo algún patrón de diseño, claro que el que tuve más claro fue el del cliente con MVC, sin embargo para el caso del servidor la cosa se me complicó bastante, ya que al comienzo simplemente seguí los tutoriales y creí que me sería más fácil identificar cual utilizar en cada caso. Sin embargo, terminé optando por hacer todo el programa de golpe y separarlo adecuadamente cuando estuviera terminado.

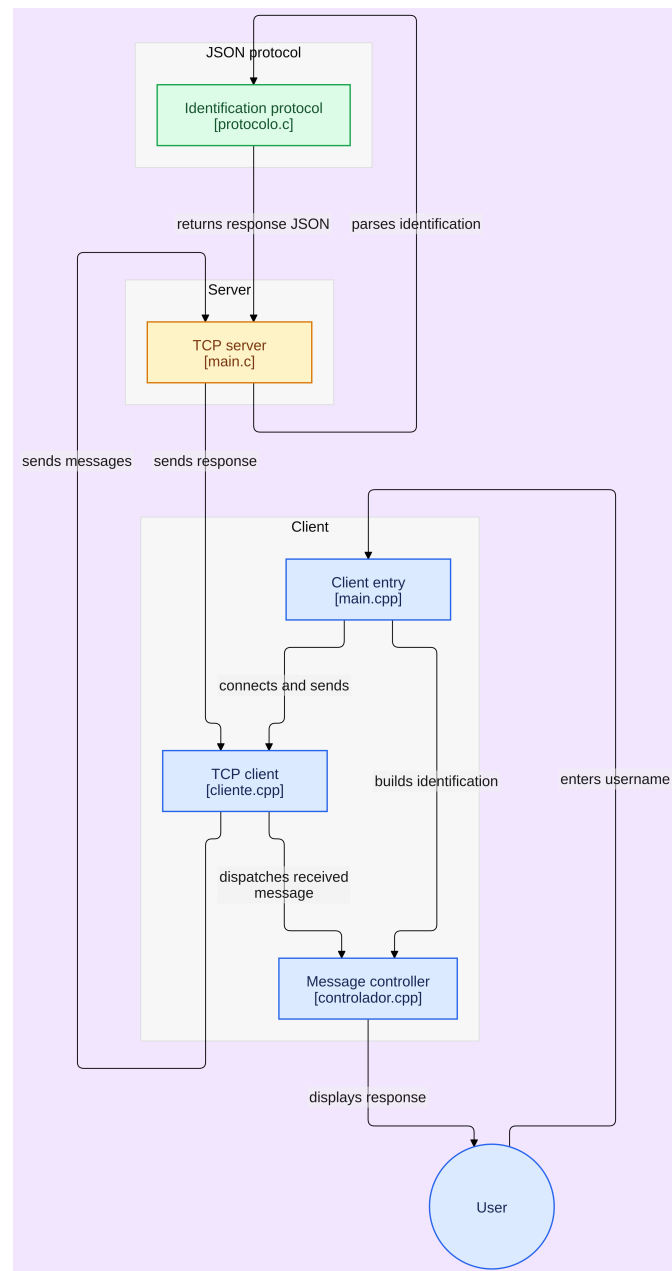
Entiendo que es una mala práctica y si bien llegué a separar unas cosas que me parecieron más sencillas de identificar porque no me convenía dejarlas juntas, en su momento creí que el loop infinito dentro del servidor sería pequeño y que solo agregaría lo indispensable. En la práctica terminé añadiendo todo porque cuando me dí cuenta que lo tuve que haber separado desde un inicio ya era un poco tardé y prioricé terminar todo antes de tratar de seguir un patrón que me facilitara las cosas.

El tener gran parte del servidor concentrada en pocos archivos y funciones hizo que eventualmente me perdiera con facilidad y se complicara modificar las cosas cuando algo se rompía o no funcionaba como lo había pensado, prácticamente utilizaba el `ctrl + f` en todo momento. Lo único bueno es que desde el comienzo fui dejando comentarios que me permitían saber que hacía cada parte, además de que los nombres de las variables los hice lo más explícitos que podía pero tratando de que fueran cortos porque me terminaría hartando de escribir un nombre largo y poco eficaz.

Eventualmente realicé distintas refactorizaciones para ayudarme a encontrar con mayor facilidad las cosas, además de corregir partes que durante las clases alguien que había hecho algo similar recibía la indicación por parte de Canek de que estaba mal hecho, así que aprovechaba para cambiarlo yo también antes de que fuera demasiado tarde.

3. Descripción general del sistema

3.1. Arquitectura cliente/servidor



4. Protocolo de comunicación

4.1. Operaciones del protocolo

Operación	Descripción
IDENTIFY	Identifica al usuario ante el servidor con un nombre.
STATUS	Cambia el estado del usuario (ACTIVE , AWAY o BUSY).
USERS	Solicita la lista de usuarios conectados y su estado.
TEXT	Envía un mensaje privado a otro usuario.
PUBLIC_TEXT	Envía un mensaje a todos los usuarios conectados.
NEW_ROOM	Crea una nueva sala, de la que el creador es el primer miembro.
INVITE	Invita a uno o más usuarios a una sala en la que ya se es miembro.
JOIN_ROOM	Une al usuario a una sala a la que fue invitado.
ROOM_USERS	Solicita la lista de usuarios de una sala a la que se pertenece.
ROOM_TEXT	Envía un mensaje a todos los miembros de una sala.
LEAVE_ROOM	Abandona una sala de la que se es miembro.
DISCONNECT	Se desconecta del chat y abandona todas sus salas.

Mensaje	Descripción
NEW_USER	Un nuevo usuario se identificó en el chat.
NEW_STATUS	Un usuario cambió su estado.
USER_LIST	Respuesta a USERS .
TEXT_FROM	Mensaje privado recibido de otro usuario.
PUBLIC_TEXT_FROM	Mensaje público recibido de otro usuario.
INVITATION	Invitación recibida a una sala.
JOINED_ROOM	Un usuario se unió a una sala.
ROOM_USER_LIST	Respuesta a ROOM_USERS .
ROOM_TEXT_FROM	Mensaje recibido dentro de una sala.
LEFT_ROOM	Un usuario abandonó una sala.
DISCONNECTED	Un usuario se desconectó del chat.
RESPONSE	Resultado de una operación solicitada.

5. Implementación

5.1. Tecnologías utilizadas

El servidor fue desarrollado en C y el cliente en C++. Para la comunicación se utilizaron sockets TCP, mientras que los mensajes intercambiados entre ambos programas se representan mediante JSON utilizando la biblioteca json-c. El servidor utiliza además GLib para el manejo de las tablas hash (diccionarios) empleadas en la administración de usuarios y salas.

Para la construcción del proyecto se utilizó Meson junto con Ninja. El cliente utiliza las herramientas de concurrencia de C++, principalmente `std::thread`, `std::mutex` y `std::condition_variable`, para recibir mensajes del servidor de forma independiente a la interacción con el usuario.

5.2. Requerimientos

Para compilar y ejecutar el proyecto se requiere un sistema compatible con sockets POSIX y contar con las siguientes herramientas y bibliotecas:

- GCC y G++.
- Meson y Ninja.
- Biblioteca `json-c`.
- Biblioteca GLib.

El proyecto está configurado para ejecutarse localmente, utilizando `127.0.0.1` como dirección del servidor y el puerto `1234` para la comunicación TCP.

5.3. Compilación y ejecución

Desde el directorio raíz del proyecto se puede realizar la configuración inicial y compilación mediante:

```
meson setup build
meson compile -C build
```

Una vez compilado, primero se debe iniciar el servidor:

```
./build/servidor/servidor
```

Posteriormente, desde otra terminal se ejecuta el cliente:

```
./build/cliente/cliente
```

El cliente solicitará el nombre de usuario y, después de establecer la conexión, permitirá utilizar las operaciones definidas en el protocolo. Los mensajes intercambiados entre cliente y servidor se procesan internamente como JSON y no forman parte de la interacción directa con el usuario.

5.4. Organización del proyecto

El código se divide en dos componentes principales. El directorio `servidor/` contiene la implementación del servidor TCP, el manejo del protocolo, las estructuras relacionadas con los clientes y la administración de las salas. El directorio `cliente/` contiene la implementación del programa cliente, incluyendo la interacción con el usuario, la construcción de mensajes y la recepción de información del servidor.

El proyecto utiliza un archivo `meson.build` en el directorio raíz para coordinar la compilación de ambos componentes.

6. Refactorización y patrones de diseño

6.1. Factory

Originalmente, crear un cliente nuevo al aceptar una conexión implicaba reservar memoria para struct cliente y para su buffer de lectura, e inicializar cada uno de sus campos (socket, estado inicial, buffer, bandera de identificación) directamente dentro del bucle principal de main(). Esta lógica se extrajo a una función fábrica, crear_cliente(socket, capacidad_buffer), que devuelve un cliente completamente inicializado o NULL si falla alguna reservación de memoria, junto con su contraparte destruir_cliente(). Con esto, deja de conocer los detalles internos de cómo se construye un cliente: solo le pide uno nuevo a la fábrica cada vez que accept() tiene éxito.

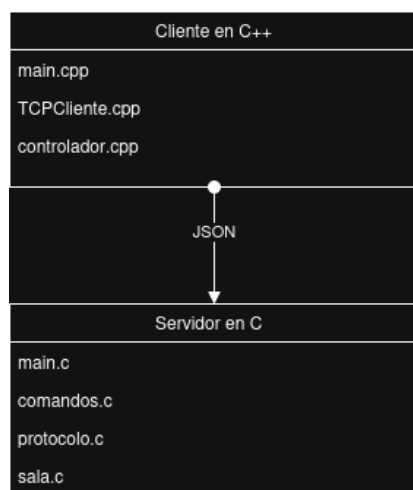
6.2. Command

El manejo de mensajes creció hasta convertirse en una larga cadena de if/else if sobre el campo "type" del mensaje, con las doce operaciones del protocolo resueltas dentro del mismo bucle de lectura de main(). Esto se refactorizó aplicando el patrón Command: cada operación se extrajo a su propia función con una firma común manejador_comando, definida en comandos.h, y se agrupó el estado que estas funciones necesitan compartir en una sola struct contexto_servidor, en vez de pasarlos como parámetros sueltos en cada llamada.

Una tabla estática en comandos.c asocia cada cadena de tipo, IDENTIFY, STATUS, etc, con su función manejadora correspondiente; manejar_mensaje() extrae el tipo del mensaje, aplica la regla de que solo IDENTIFY es válido antes de que el cliente se identifique, y busca en la tabla la función a invocar. Agregar una operación nueva al protocolo implica escribir una función más y añadir una fila a la tabla, en vez de otra rama en una cadena de if/else cada vez más larga. Esta refactorización fue la más importante ya que me daba asco como se vea completa en el main.c, se verificó contra pruebas del protocolo para no alterar el comportamiento ya implementado.

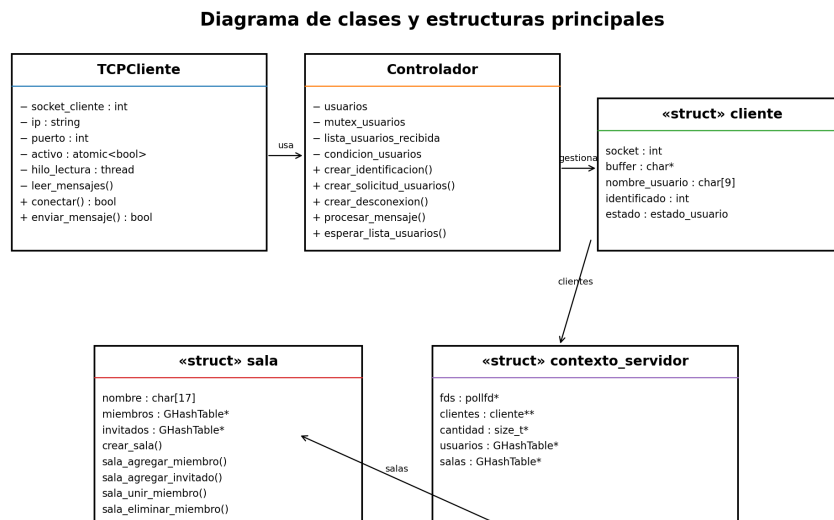
7. Diagramas UML

7.1. Diagrama de componentes



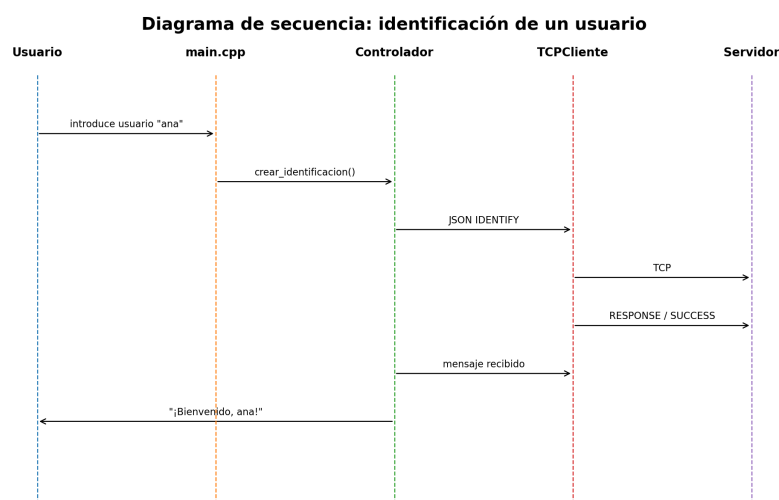
Esto representa como funciona cada parte a grandes rasgos, ya que en arquitectura esta un diagrama más específico de todo lo que hace el proyecto.

7.2. Diagrama de clases



Este diagrama muestra las principales clases y estructuras utilizadas en el proyecto, junto con sus responsabilidades y relaciones. Permite observar cómo se organiza internamente el cliente y el servidor y qué función cumple cada elemento dentro del sistema.

7.3. Diagrama de secuencia



Este diagrama muestra la secuencia de comunicación entre el cliente y el servidor durante una operación del sistema. Se pueden observar los mensajes que se intercambian, el procesamiento realizado por el servidor y la respuesta que recibe el cliente. También permite representar el funcionamiento del hilo encargado de recibir los mensajes en el cliente.

8. Problemas encontrados y soluciones

8.1. Problema 1

El primer gran problema fue que no sabía utilizar GitHub y cada vez que hacía commits me pedía mi usuario y contraseña.

8.1.1. Solución

Afortunadamente Miriam mandó al telegram un tutorial para solucionar esto y me permitió hacer commits de una forma más eficiente.

8.2. Problema 2

Escoger el lenguaje de programación C.
En algún punto sentía que por más código que escribía no estaba haciendo realmente nada y me sentía bastante frustrado porque era culpa del lenguaje de programación que escogí.

8.2.1. Solución

No hay solución, desde hoy soy hater #1 de C y no pienso volver a utilizarlo nunca más.

8.3. Problema 3

No sabía exactamente como comenzar y los tutoriales eran poco claros, además la parte del servidor era la que más dudas me generaba.

8.3.1. Solución

Tuve que buscar más referencias y empezar a combinar todas las cosas que veía que podían ser útiles para mi proyecto, ya que no todo el código que utilizaba de referencia por parte de los repositorios me era útil.

8.4. Problema 4

Me costó bastante la parte de JSON. Era algo completamente nuevo y hubo muchas veces que no salían los mensajes como lo planeaba, esto no se llegó a reflejar tanto porque solo hacía commit cuando compilaba, pero creo que todo lo relacionado a JSON fue bastante complicado al comienzo.

8.4.1. Solución

Prueba y error, además de tomar bastante de guía los ejemplos que venían en el protocolo.

8.5. Problema 5

Saturar mi main. Creo que fue el problema más grande que llegué a tener, conforme iba avanzando veía que el main crecía más y aunque traté de separarlo no llegaba a encontrar el momento adecuado y sentía que si lo movía se iba a romper todo, así que prioricé terminar para después refactorizar.

8.5.1. Solución

Leí la página que compartió Miriam y utilicé la más conveniente en mi caso para así dejar el main más limpio.

8.6. Problema 6

Creaba copias de las listas de usuarios en lugar de utilizar la original con ayuda de los apuntadores.

8.6.1. Solución

Esta fue complicado pero no tanto, ya que Canek mencionó en la clase como debíamos abordar este problema y justamente acababa de implementarlo, entonces no me fue difícil cambiar todo antes de que fuera demasiado tarde.

9. Conclusiones

Al momento de comenzar creí que tendría tiempo suficiente para terminarlo, sin embargo, me dí cuenta que era bastante más pesado de completar de lo que había imaginado. En un inicio tenía pensado hacer interfaz gráfica y hacer tests, pero conforme avanzaba me quede sin tiempo y tuve que desvelarme para terminar a tiempo con la fecha de entrega del viernes.

Al final fue un proyecto en el que tuve que investigar muchas cosas y algunas otras copiarlas porque no sabía exactamente como implementarlas, sin embargo, hubo un punto en el que era más copiar y pegar de las cosas que ya tenía y eso facilito bastante a la hora de terminar con todo lo que pedía el protocolo.

Aprendí que a C no se le tiene miedo, se le tiene respeto.

10. Referencias

- Deitel, H. M., & Deitel, P. J. (2004). *Cómo programar en C, C++ y Java*. Prentice Hall.
- Beej's Guide to Network Programming: <https://beej.us/guide/bgnet0/>
- Building a Simple TCP Chat Application in C: <https://medium.com/@trish07/building-a-simple-tcp-chat-application-in-c-a-step-b>
- Understanding TCP and Building our Own TCP Server in C: https://medium.com/@shivambhadani_/understanding-tcp-and-building-our-own-tcp-se

- IBM Documentation, *Using poll() instead of select()*:
<https://www.ibm.com/docs/en/i/7.4.0?topic=designs-using-poll-instead-select>
- AmineToualbi, *TCPChat* (repositorio de referencia):
<https://github.com/AmineToualbi/TCPChat.git>
- BaseMax, *TCP-Chat-CPP* (repositorio de referencia):
<https://github.com/BaseMax/TCP-Chat-CPP.git>
- JimmyWorks, *TCP-Chat-App* (repositorio de referencia):
<https://github.com/JimmyWorks/TCP-Chat-App.git>
- JSON, *JSON.org*: <https://www.json.org/json-es.html>
- Documentación oficial de json-c:
<https://github.com/json-c/json-c> (API: <https://json-c.github.io/json-c/>)
- Documentación oficial de GLib:
<https://docs.gtk.org/glib/>